

Monte Carlo Simulation of Annealing and Grain Growth

Raisya Jasmine Zahira

Jonathan Leewin

May 26, 2025

Abstract

A Python Monte Carlo model of the annealing process in polycrystalline materials is shown in this report. The simulation has mechanisms for rate sensitivity, specific component soaking, sudden cooling effects, locating improper grain and determining what stresses remain. The progress of grain development and energy is modeled by a 2D grid that makes use of the Potts model and the Metropolis algorithm. The simulation results are presented using graphs of energy, temperature and microstructures, along with statistical data on grain size distributions. Both students and researchers can use the modular codebase to learn about annealing processes in materials science.

Contents

1	Introduction	3
2	Project Overview and Architecture	3
2.1	Key Features	3
2.2	Project Structure	4
2.3	Installation and Setup	4
2.4	Basic Usage	5
2.5	Troubleshooting Common Issues	5
3	Mathematical Foundation	5
3.1	The Potts Model	5
3.2	Extended Energy Model with Residual Stress	6
3.3	Metropolis Algorithm	6
3.4	Mobility-Modified Metropolis Algorithm	6
3.5	Grain Growth Kinetics	7
4	Simulation Methodology	7
4.1	Grid Initialization	7
4.2	Monte Carlo Evolution	8
4.3	Temperature Control	8
4.4	Statistical Analysis	8

5	Simulation Parameters	9
5.1	Material Properties	9
6	Results and Analysis	9
6.1	Energy Evolution	9
6.2	Microstructural Evolution	11
6.3	Grain Size Distribution Analysis	11
6.4	Residual Stress Analysis	12
7	Key Implementation Details	12
7.1	Monte Carlo Step Implementation	12
7.2	Initialization Algorithm	13
7.3	Energy Calculation	13
8	Advanced Features	14
8.1	Abnormal Grain Detection	14
8.2	Component-Specific Soaking	14
8.3	Rate Sensitivity Analysis	14
9	Challenges and Lessons Learned	14
9.1	Technical Challenges	14
9.2	Physical Modeling Limitations	15
9.3	Key Insights	15
10	Future Work and Recommendations	15
10.1	Immediate Improvements	15
10.2	Advanced Features	16
10.3	Educational Applications	16
11	Conclusion	16
12	Acknowledgments	17

1 Introduction

Materials engineers use annealing as a necessary heat treatment to transform the microstructure of polycrystalline materials. To treat the material, it is heated to a high temperature, kept at that heat for a set amount of time (soaking) and cooled under measurement (controlled cooling). The changes made during annealing contain grain growth, reducing stress and reorganizing the crystal structures, resulting in increased ductility, softer material and relief from remaining stresses.

The basic reason grain size increases during annealing is to decrease the total area of grain boundaries and associated system energy. While grains grow, the system's overall interfacial energy decreases which increases its stability. Because material properties are vital in making products, this process is highly required in manufacturing.

Here, a Monte Carlo simulation together with a 2D microstructural grid are applied to model the growth of grains in an annealing process. The simulation includes a number of advanced tools that make it different from basic approaches.

- Advanced energy models including residual stress contributions
- Material-specific properties and temperature-dependent behavior
- Component-specific soaking time calculations
- Abnormal grain detection and tracking
- Comprehensive statistical analysis and visualization
- Modular architecture for extensibility and research applications

2 Project Overview and Architecture

2.1 Key Features

- **Monte Carlo Simulation:** Implementation of grain growth using the Potts model and Metropolis algorithm
- **Material Properties:** Support for multiple materials with temperature-dependent properties
- **Advanced Temperature Control:** Heating, soaking, and cooling phases with configurable profiles
- **Residual Stress Modeling:** Prediction and visualization of stress fields during annealing
- **Grain Analysis:** Statistical tracking of grain size distributions and abnormal grain detection

- **Comprehensive Visualization:** Real-time and post-processing visualization capabilities
- **Extensible Architecture:** Modular design supporting research and educational applications

2.2 Project Structure

The codebase follows a modular architecture that separates concerns and promotes maintainability:

```
1 annealing/  
2     src/  
3         models/  
4             annealing_simulation.py  
5             material_properties.py  
6             monte_carlo.py  
7             temperature_controller.py  
8         analysis/  
9             grain_analyzer.py  
10        visualization/  
11            visualizer.py  
12        utils/  
13            helpers.py  
14        main.py  
15    frames/  
16    results/  
17    requirements.txt  
18    README.md
```

Listing 1: Project directory structure

2.3 Installation and Setup

To set up the simulation environment:

1. Clone the repository:

```
1 git clone https://github.com/L1M1N4L/comp-phys-final-annealing  
2 cd annealing
```

2. Create virtual environment (recommended):

```
1 python -m venv venv  
2 source venv/bin/activate # On Windows: venv\Scripts\activate
```

3. Install dependencies:

```
1 pip install -r requirements.txt
```

2.4 Basic Usage

The simplest way to run the simulation is:

```
1 python src/main.py
```

This command will:

1. Initialize the simulation with default parameters
2. Execute the complete annealing process (heating, soaking, cooling)
3. Generate comprehensive visualizations in the **frames/** directory
4. Save numerical results in the **results/** directory
5. Display inspection results and statistics in the console

2.5 Troubleshooting Common Issues

- **No visible grain growth:** Reduce the number of initial grains or increase the annealing temperature. The system may be frozen due to low thermal energy.
- **Missing attribute errors:** Ensure you're using the latest version and only running `main.py` for basic simulations.
- **Dependency issues:** Verify all packages in `requirements.txt` are properly installed.
- **Performance problems:** Consider reducing grid size or number of Monte Carlo steps for faster execution.
- **Visualization errors:** Check that output directories exist and have write permissions.

3 Mathematical Foundation

3.1 The Potts Model

The simulation is centered on the q -state Potts model which gives a statistical mechanical approach to understand growth in grain boundaries of polycrystalline materials. In this method, the sites on a 2D lattice are given states (grain orientations) selected from a set of q options.

The total system energy is expressed as:

$$E = \sum_{\langle i,j \rangle} J_{ij}(1 - \delta_{s_i, s_j}) \quad (1)$$

where:

- $\langle i, j \rangle$ denotes summation over nearest-neighbor pairs

- J_{ij} is the interaction energy between sites i and j
- δ_{s_i, s_j} is the Kronecker delta function
- s_i represents the state (grain orientation) at site i

Grain boundary surfaces are formulated as the places where s_i and s_j differ, just as physics considers them to be energy-rich parts.

3.2 Extended Energy Model with Residual Stress

To capture more realistic material behavior, the energy model is extended to include residual stress contributions:

$$\Delta E = E_{new} - E_{old} + \sigma_{residual}(T, \dot{\epsilon}) \quad (2)$$

The residual stress term depends on temperature T and strain rate $\dot{\epsilon}$:

$$\sigma_{residual} = \sigma_0 \exp\left(-\frac{Q}{RT}\right) \left(\frac{\dot{\epsilon}}{\dot{\epsilon}_0}\right)^m \quad (3)$$

where:

- σ_0 is a reference stress
- Q is the activation energy for stress relaxation
- R is the gas constant
- $\dot{\epsilon}_0$ is a reference strain rate
- m is the strain rate sensitivity exponent

3.3 Metropolis Algorithm

The Metropolis algorithm is used to decide on new microstructural changes being proposed by the system. For a proposed transition with energy change ΔE , the acceptance probability is:

$$P_{accept} = \begin{cases} 1 & \text{if } \Delta E \leq 0 \\ \exp\left(-\frac{\Delta E}{k_B T}\right) & \text{if } \Delta E > 0 \end{cases} \quad (4)$$

where k_B is the Boltzmann constant and T is the absolute temperature.

3.4 Mobility-Modified Metropolis Algorithm

For more sophisticated modeling, the acceptance probability can be modified by a mobility factor $M(T)$:

$$P_{accept} = \begin{cases} M(T) & \text{if } \Delta E \leq 0 \\ M(T) \exp\left(-\frac{\Delta E}{k_B T}\right) & \text{if } \Delta E > 0 \end{cases} \quad (5)$$

The temperature-dependent mobility typically follows an Arrhenius relationship:

$$M(T) = M_0 \exp\left(-\frac{Q_m}{k_B T}\right) \quad (6)$$

where Q_m is the activation energy for grain boundary migration.

3.5 Grain Growth Kinetics

The evolution of average grain size follows the general kinetic law:

$$\bar{D}^n - \bar{D}_0^n = kt \quad (7)$$

where:

- $\bar{D}$ is the average grain diameter at time t
- $\bar{D}_0$ is the initial average grain diameter
- n is the growth exponent (typically 2 for normal grain growth)
- k is the temperature-dependent rate constant
- t is the annealing time

The rate constant exhibits Arrhenius temperature dependence:

$$k(T) = k_0 \exp\left(-\frac{Q_{growth}}{RT}\right) \quad (8)$$

For detailed numerical implementation, parameter optimization studies, and extended mathematical derivations, visit:

GitHub Repository: <https://github.com/L1M1N4L/comp-phys-final-annealing>

4 Simulation Methodology

4.1 Grid Initialization

In the beginning, the simulation develops on a $N \times N$ square lattice. Every lattice site reflects a minuscule region in the material and carries a numerical index that indicates its grain orientation. The process of initialization covers the following tasks:

1. Creating a uniform random distribution of grain orientations across the lattice
2. Ensuring that the specified number of unique grains is present
3. Establishing initial grain centers to promote realistic grain morphologies

The initialization algorithm ensures diversity in the initial microstructure while maintaining statistical uniformity.

4.2 Monte Carlo Evolution

Each Monte Carlo step consists of N^2 attempted spin flips, where N is the linear dimension of the lattice. The algorithm proceeds as follows:

1. **Site Selection:** Randomly select a lattice site (i, j)
2. **Neighbor Analysis:** Identify the orientations of nearest neighbors
3. **Transition Proposal:** Randomly select a neighbor's orientation as the proposed new state
4. **Energy Calculation:** Compute the energy change ΔE for the proposed transition
5. **Acceptance Decision:** Apply the Metropolis criterion to accept or reject the change
6. **State Update:** If accepted, update the site's orientation; otherwise, retain the current state

4.3 Temperature Control

The annealing process is simulated through a carefully controlled temperature profile consisting of three phases:

1. **Heating Phase:** Linear temperature increase from ambient to soaking temperature
2. **Soaking Phase:** Isothermal holding at the target temperature for a specified duration
3. **Cooling Phase:** Controlled temperature decrease back to ambient conditions

The temperature at each time step is given by:

$$T(t) = \begin{cases} T_0 + r_{heat} \cdot t & \text{heating phase} \\ T_{soak} & \text{soaking phase} \\ T_{soak} - r_{cool} \cdot (t - t_{soak}) & \text{cooling phase} \end{cases} \quad (9)$$

where r_{heat} and r_{cool} are the heating and cooling rates, respectively.

4.4 Statistical Analysis

Throughout the simulation, comprehensive statistics are collected including:

- Total system energy evolution
- Number of unique grains as a function of time
- Grain size distribution histograms
- Average grain size and standard deviation
- Detection and tracking of abnormally large grains
- Spatial correlation functions

5 Simulation Parameters

Table 1 summarizes the key parameters used in the simulation, along with their physical significance and typical values.

Parameter	Symbol	Typical Value	Physical Significance
Grid size	N	100	Spatial resolution of simulation
Initial grains	n_{grains}	50	Initial microstructural complexity
Max iterations	n_{steps}	500	Total simulation time
Initial temperature	T_{init}	300 K	Ambient starting temperature
Soak temperature	T_{soak}	1000 K	Processing temperature
Final temperature	T_{final}	300 K	Final ambient temperature
Heating rate	r_{heat}	5 K/step	Heating ramp rate
Soak duration	t_{soak}	100 steps	Isothermal holding time
Cooling rate	r_{cool}	0.5 K/step	Cooling ramp rate
Boundary energy	J	1.0	Grain boundary energy density
Boltzmann constant	k_B	1.0	Statistical mechanics constant

Table 1: Key simulation parameters and their typical values.

5.1 Material Properties

The simulation supports multiple materials with distinct properties. Table 2 shows examples of material-specific parameters.

Property	Steel	Aluminum	Copper
Melting point (K)	1811	933	1358
Grain boundary energy (J/m ²)	1.0	0.6	0.8
Activation energy (kJ/mol)	240	140	180
Mobility factor	1.0	1.5	1.2
Thermal expansion ($\times 10/K$)	12	23	17

Table 2: Material-specific properties used in the simulation.

6 Results and Analysis

6.1 Energy Evolution

The simulation demonstrates characteristic energy evolution during annealing. The energy plot shows several key features:

- **Initial rapid decrease:** Corresponding to elimination of high-energy configurations

- **Gradual decline during heating:** As thermal activation enables grain boundary migration
- **Plateau during soaking:** Indicating approach to quasi-equilibrium
- **Final stabilization:** As the system reaches a lower energy state

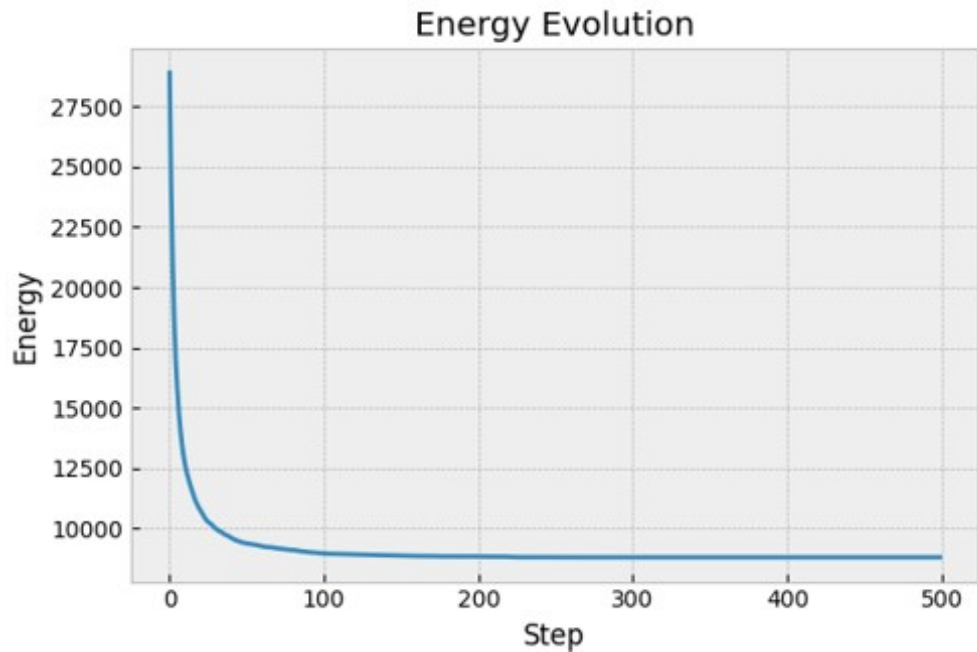


Figure 1: Energy evolution during the annealing process

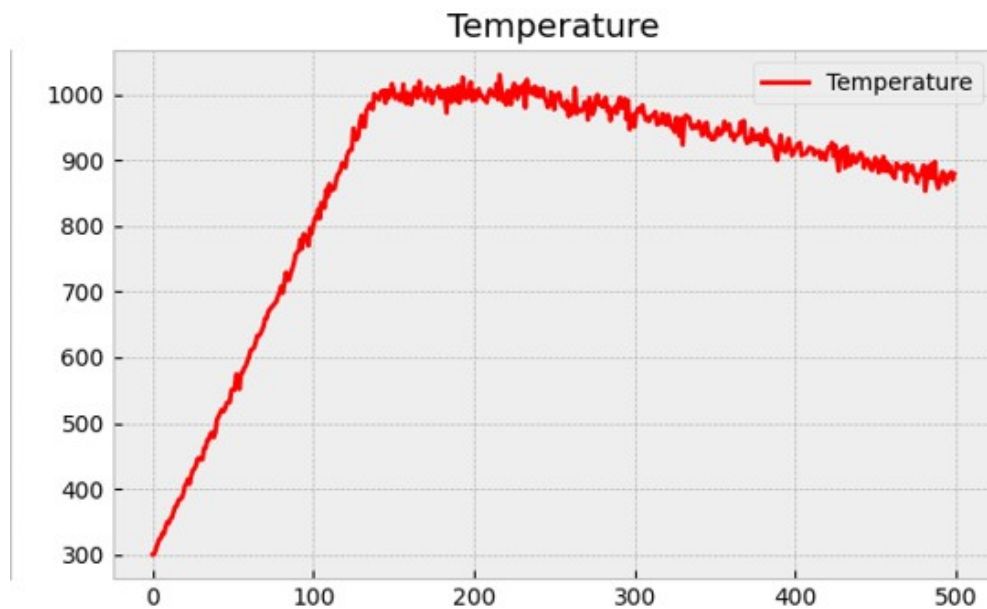


Figure 2: Temperature evolution during the annealing process

6.2 Microstructural Evolution

As annealing occurs, the microstructure goes from a lot of little grains to having fewer, bigger grains that are created during annealing.

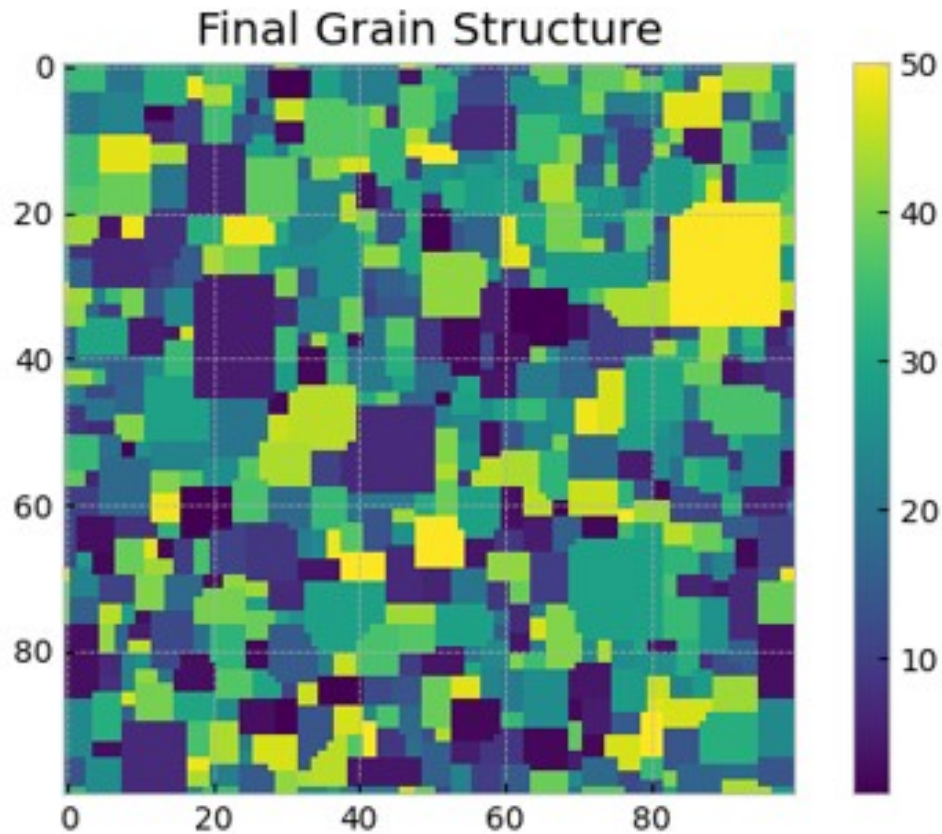


Figure 3: Final Grain Structure during the annealing process

6.3 Grain Size Distribution Analysis

After running the simulation, we review the grain size distribution and can see key statistics about the resulting microstructure, including its shape and skewness, the presence of unusually large grains, average grain size and how the actual distribution measures up to what is expected in a perfect simulation.

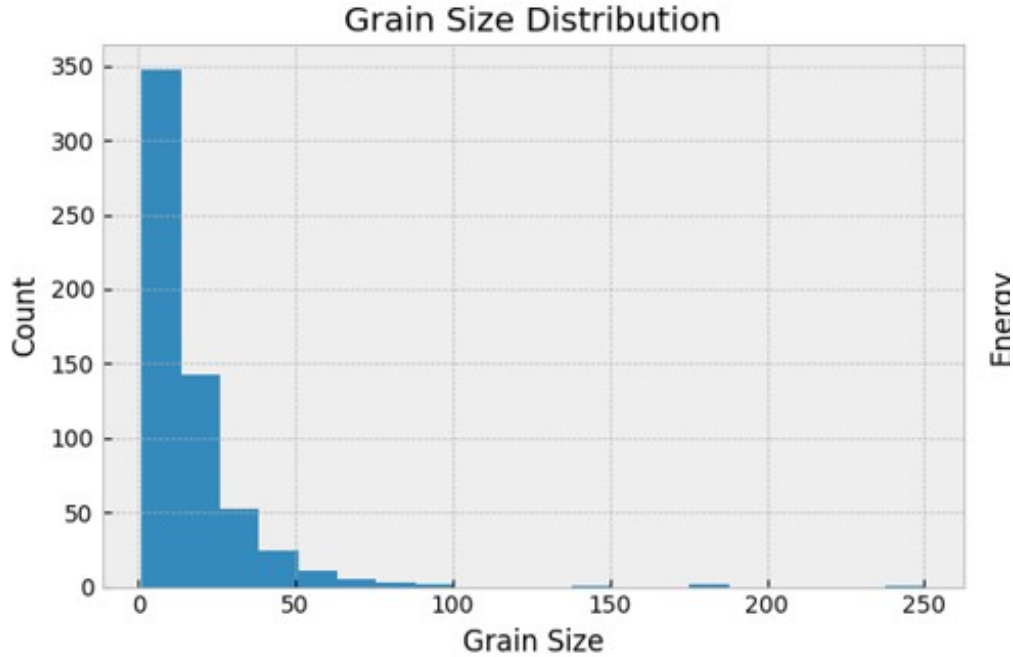


Figure 4: Histogram for the Grain size distribution

6.4 Residual Stress Analysis

The simulation predicts residual stress fields that develop during annealing due to thermal expansion mismatches and grain boundary constraints. These stress fields provide insight into stress concentration near grain boundaries, evolution of stress during temperature cycling, relationship between grain size and local stress, and implications for material performance.

7 Key Implementation Details

7.1 Monte Carlo Step Implementation

The core Monte Carlo algorithm is implemented as follows:

```

1 def monte_carlo_step(self, grain_analyzer):
2     """Execute one complete Monte Carlo step."""
3     self.temperature = self.temperature_controller.update_temperature()
4     attempts = self.grid_size * self.grid_size
5
6     for _ in range(attempts):
7         # Random site selection
8         i = np.random.randint(0, self.grid_size)
9         j = np.random.randint(0, self.grid_size)
10        current = self.grid[i, j]
11
12        # Neighbor analysis
13        neighbors = [

```

```

14         self.grid[(i+di)%self.grid_size, (j+dj)%self.grid_size]
15         for di, dj in [(-1,0), (1,0), (0,-1), (0,1)]
16     ]
17     new_orientation = neighbors[np.random.randint(0, len(neighbors))]
18
19     # Energy calculation
20     old_energy = self.calculate_energy(i, j)
21     self.grid[i, j] = new_orientation
22     new_energy = self.calculate_energy(i, j)
23     delta_energy = new_energy - old_energy
24
25     # Metropolis criterion
26     if delta_energy > 0:
27         kT = self.boltzmann_constant * self.temperature
28         if kT == 0 or np.random.random() > np.exp(-delta_energy / kT):
29             self.grid[i, j] = current # Reject change
30
31     self._update_statistics(grain_analyzer)

```

Listing 2: Monte Carlo step implementation

7.2 Initialization Algorithm

The grid initialization ensures proper distribution of grains:

```

1 def create_initial_structure(self, num_grains):
2     """Create initial grain structure with specified number of grains."""
3     # Random assignment of grain orientations
4     self.grid[:, :] = np.random.randint(
5         1, num_grains + 1,
6         size=(self.grid_size, self.grid_size),
7         dtype=np.int32
8     )
9
10    # Ensure all grain orientations are present
11    positions = np.random.permutation(
12        self.grid_size * self.grid_size
13    )[:num_grains]
14    x = positions % self.grid_size
15    y = positions // self.grid_size
16    self.grid[y, x] = np.arange(1, num_grains + 1, dtype=np.int32)

```

Listing 3: Grid initialization

7.3 Energy Calculation

The local energy calculation considers nearest-neighbor interactions:

```

1 def calculate_energy(self, i, j):
2     """Calculate local energy at site (i,j)."""
3     current = self.grid[i, j]
4     energy = 0

```

```

5
6     # Check all nearest neighbors
7     for di, dj in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
8         ni, nj = (i + di) % self.grid_size, (j + dj) % self.grid_size
9         neighbor = self.grid[ni, nj]
10
11     # Add energy for grain boundary
12     if current != neighbor:
13         energy += self.boundary_energy
14
15     return energy

```

Listing 4: Local energy calculation

8 Advanced Features

8.1 Abnormal Grain Detection

Included in the simulation are algorithms to spot and track larger grains that can develop during annealing by checking grain sizes, recording how fast the grains grow over time, examining their spatial correlation and seeing how these grains influence the microstructure.

8.2 Component-Specific Soaking

In industrial situations, the simulation calculates the appropriate soaking time for each part using its shape, thermal conductivity, final grain size and the production features.

8.3 Rate Sensitivity Analysis

Incorporated in the simulation are effects depending on strain rate in the energy model, factors that depend on temperature for easy atom movement, schedule-based conditions at the boundaries and statistical approaches to investigate rates.

9 Challenges and Lessons Learned

9.1 Technical Challenges

Several significant challenges were encountered during development:

- **Parameter Sensitivity:** The simulation exhibits high sensitivity to temperature and boundary energy parameters. Small changes can dramatically affect grain growth behavior.
- **Computational Performance:** Large grids and extended simulation times require careful optimization to achieve reasonable execution speeds.

- **Statistical Convergence:** Ensuring sufficient statistical sampling while maintaining computational efficiency.
- **Initialization Effects:** The initial grain configuration significantly impacts the final results, requiring careful initialization strategies.

9.2 Physical Modeling Limitations

- **2D Approximation:** Real materials are three-dimensional, and 2D simulations may miss important geometric effects.
- **Simplified Energy Model:** The Potts model assumes uniform grain boundary energy, while real materials exhibit anisotropic properties.
- **Neglected Phenomena:** Effects such as texture evolution, precipitate interactions, and deformation are not included.

9.3 Key Insights

- **Temperature Control is Critical:** Precise temperature control is essential for realistic grain growth simulation.
- **Statistical Analysis is Essential:** Single simulations can be misleading; ensemble averages provide more reliable results.
- **Modular Design Pays Dividends:** The modular architecture greatly facilitates debugging, testing, and extension.
- **Visualization is Invaluable:** Real-time visualization helps identify problems and understand system behavior.

10 Future Work and Recommendations

10.1 Immediate Improvements

- **3D Extension:** Implement three-dimensional grain growth simulation
- **Anisotropic Properties:** Include crystallographic texture and anisotropic grain boundary energies
- **Multi-Phase Systems:** Extend to materials with multiple phases or precipitates
- **Parallel Implementation:** Utilize parallel computing for improved performance

10.2 Advanced Features

- **Machine Learning Integration:** Use ML techniques for parameter optimization and pattern recognition
- **Experimental Validation:** Compare results with experimental data from real annealing processes
- **Multi-Scale Modeling:** Couple with larger-scale process models
- **Real-Time Control:** Develop adaptive algorithms for process optimization

10.3 Educational Applications

- Interactive GUI for educational demonstrations
- Pre-configured example cases for different materials
- Integration with materials science curricula
- Online simulation platform development

11 Conclusion

The project achieves its goal by fully applying Monte Carlo simulations to study annealing in polycrystalline materials. Through Potts model and Metropolis algorithm, the simulation reflects the important physics in grain growth and has functionalities for predicting residual stress, spotting abnormal grains and handling material-related aspects.

Because the system is built in a modular way and its code is fully documented, it works well for studies and teaching. However, the `main.py` script provides a straightforward interface for standard simulations.

Among the primary achievements are physically-based grain growth simulation, extensive statistical analysis and visualization tools, flexible design for various materials and processing, clear documentation, a user-friendly interface and recognition of important challenges and what is needed for further growth.

Though the model simulates energy reduction, more improvements and settings adjustments are required to accurately show grain coarsening during annealing. This implementation gives a good foundation for us to make improvements.

The project supports the broader aims of computational materials science by making it easier and clearer for people to study essential parts of materials engineering. Mathematical rigor, practical use and an educational perspective are why this work is important for both study and education.

12 Acknowledgments

Thanks to over thirty years of advancements in computational materials science and Monte Carlo methods, much of this work is possible. We pay tribute to researchers who first created the theoretical ideas of the Potts model and used it for grain growth simulation. Special thanks are due to the Python community for producing the libraries that made our implementation work. We also thank Mr. Tri for selecting the topic and for guiding us during the project.

References

- [1] Porter, D. A., Easterling, K. E., & Sherif, M. Y. (2009). *Phase Transformations in Metals and Alloys*. CRC Press.
- [2] Srolovitz, D. J., Anderson, M. P., Sahni, P. S., & Grest, G. S. (1984). Computer simulation of grain growth—I. Kinetics. *Acta Metallurgica*, 32(5), 793-802.
- [3] Rollett, A. D., Humphreys, F. J., Rohrer, G. S., & Hatherly, M. (2004). *Recrystallization and Related Annealing Phenomena*. Elsevier.
- [4] Chen, L. Q. (2002). Phase-field methods and micromechanics of interfaces. *Annual Review of Materials Research*, 32(1), 113-140.
- [5] Humphreys, F. J., & Hatherly, M. (2012). *Recrystallization and Related Annealing Phenomena*. Elsevier.
- [6] Callister Jr, W. D., & Rethwisch, D. G. (2018). *Materials Science and Engineering: An Introduction*. John Wiley & Sons.
- [7] Reed-Hill, R. E., Abbaschian, R., & Abbaschian, L. (2009). *Physical Metallurgy Principles*. Cengage Learning.